# 🚀 STARVNT — EVENT COMMERCIAL INTELLIGENCE & FINANCIAL LIFECYCLE MASTER DEVELOPER PROMPT

Product & Engineering Direction • Outcome-Verified Event Operating System

## To: STARVNT Product & Engineering Team

## Product Context

STARVNT is being built as an:

“Outcome-Verified Event Operating System”

STARVNT is not a generic accounting software, ERP, debt-collection platform, or fintech product.

However, an event has a complete commercial lifecycle, and STARVNT must understand that lifecycle as part of the Event OS.

The system must eventually understand:

What the event needs → what was quoted → what was booked → what was paid → what is due → what was executed → what was verified → what should be settled → what the event ultimately cost.

Therefore, we are introducing a Commercial Intelligence / Financial Control capability inside the Event OS.

This is a product architecture direction, not a standalone fintech product.

## 1. CORE EVENT LIFECYCLE

```
DISCOVER ↓ PLAN ↓ BUDGET ↓ REQUIREMENTS ↓ QUOTES ↓ BOOK ↓ PAY ↓ EXECUTE ↓ VERIFY ↓ SETTLE ↓ EXPERIENCE ↓ REMEMBER ↓ INSPIRE
```

Commercial intelligence sits across this lifecycle.

Financial lifecycle:

BUDGET → QUOTE → BOOKING → PAYMENT → PAYMENT DUE → EXECUTION → COMPLETION → VERIFICATION → SETTLEMENT → EVENT PROFITABILITY / FINANCIAL OUTCOME

## 2. IMPORTANT ARCHITECTURAL PRINCIPLE

Do NOT create a separate financial universe disconnected from Event Core.

Financial state must remain connected to:

Event / Requirement / Vendor / Quote / Booking / Payment / Execution / Completion / Verification / Settlement

Example:

```
Event ↓ Photography Requirement ↓ Photography Vendor ↓ Quote = ₹50,000 ↓ Booking = Confirmed ↓ Paid = ₹20,000 ↓ Remaining = ₹30,000 ↓ Execution = Scheduled ↓ Completion = Pending ↓ Verification = Pending ↓ Settlement = Not Eligible
```

## 3. NEVER CONFUSE THESE STATES

```
Booking = Customer has accepted/reserved the service. Payment = Money has actually been received and verified. Execution = The service is being delivered. Completion = Vendor has claimed the service is completed. Verification = STARVNT has verified the agreed outcome/service. Settlement = Vendor becomes eligible for settlement.
```

Booking ≠ Payment ≠ Completion ≠ Verification ≠ Settlement

This separation must exist at the data/state level, not only in UI.

## 4. FINANCIAL TRUTH LAYER

> Event level: Total Budget / Committed Amount / Paid Amount / Upcoming Amount / Remaining Amount / Overdue Amount / Projected Total
>
> Booking level: Quoted Amount / Booked Amount / Amount Paid / Amount Due / Payment Due Date / Payment Status / Execution Status / Completion Status / Verification Status / Settlement Status
>
> Vendor level: Expected Receivables / Received / Due / Overdue / Upcoming / Expected Settlement / Settled

Derive these values from authoritative Core data. Do not create multiple independent calculations in different dashboards.

## 5. CUSTOMER FINANCIAL EXPERIENCE

Customer should NOT feel like they are using accounting software.

My Event Financial Picture

> Total Budget: ₹15,00,000 Committed: ₹10,50,000 Paid: ₹6,20,000 Upcoming: ₹2,80,000 Remaining: ₹4,50,000

What Needs Your Attention?

> Photography payment due in 2 days. [Pay Now] Decoration advance pending. [View Details] Catering payment scheduled for 20 November. [View Payment]
>
> Customer goal: “Where does my event stand financially, and what do I need to do next?”

## 6. VENDOR COMMERCIAL EXPERIENCE

> Expected Receivables: ₹4.80L Due This Week: ₹1.20L Overdue: ₹82K Expected Settlements: ₹2.10L
>
> Potential aging: Due Today / 1–7 Days / 8–30 Days / 31–60 Days / 60+ Days

Vendor should eventually understand who owes money, how much, when it was due, missed commitments, pending payments, bookings awaiting payment, and completed services awaiting settlement.

But this must remain event-centric.

## 7. AURA+ FINANCIAL INTELLIGENCE

Aura+ must NOT become the source of financial truth.

> Core says: ₹82,000 overdue.

Aura+ interprets it.

> Vendor asks: “□□□□ □□□ □□□ customer-□□□ follow-up □□□ □□□□?”
>
> Aura+: “Three customers need attention. One has ₹82,000 overdue for 52 days, another has ₹35,000 due today, and one has a missed commitment date.”
>
> Then: “Would you like to review the priority list?”

This is Next Best Action.

> Aura+ responsibilities: Understand → Analyze → Prioritize → Explain → Recommend → Assist

Aura+ must never invent financial values. All financial facts must originate from authoritative Core/API data.

## 8. PAYMENT COMMUNICATION PHILOSOPHY

STARVNT should facilitate payments, not become an aggressive debt collector.

Good: Payment due tomorrow. Your payment link is ready. Your next milestone payment is pending. Your commitment date has passed. Review payment.

Avoid: PAY NOW!!! You haven't paid yet. Hurry! We will keep calling you. Pay immediately or else.

## 9. PAYMENT → VERIFICATION → CORE STATE

Customer Payment ↓ Payment Gateway ↓ Webhook / Verification ↓ Payment Verification ↓ Core Financial State ↓ Booking / Event State Update

Never rely solely on frontend payment success. Server-side verification is mandatory.

## 10. PAYMENT LEDGER

Event └── Booking ├── Total Amount ├── Payment Schedule ├── Payments ├── Amount Paid ├── Amount Due ├── Due Dates ├── Payment Status └── Verification Status

Every financial state change should be traceable.

₹20,000 Payment Initiated ↓ Gateway Confirmation ↓ Webhook Received ↓ Payment Verified ↓ Ledger Updated ↓ Booking Financial State Updated

## 11. PAYMENT STATUS MODEL

Conceptual states:

PENDING / INITIATED / PROCESSING / SUCCESS / FAILED / REFUNDED / PARTIALLY_PAID / OVERDUE / CANCELLED

Exact enum naming may follow existing STARVNT conventions. Do not collapse everything into Paid / Unpaid.

## 12. PAYMENT SCHEDULE

Bookings should eventually support milestone-based payments.

Photography — Total ₹50,000

Advance: ₹20,000 — Paid ✓ Milestone 2: ₹15,000 — Due 15 Nov Final: ₹15,000 — Due at Event completion

Foundation for reminders, payment links, financial visibility, overdue detection, settlement logic.

## 13. EXECUTION + FINANCIAL STATE

BOOKED ↓ SCHEDULED ↓ CHECK-IN ↓ IN PROGRESS ↓ COMPLETION SUBMITTED ↓ EVIDENCE ↓ VERIFICATION ↓ COMPLETED ↓ SETTLEMENT ELIGIBLE ↓ SETTLED

Actual states may follow existing STARVNT Core lifecycle, but commercial system must consume those states.

## 14. VERIFICATION REGISTRY

Vendor says completed ≠ STARVNT verified completed.

STARVNT must eventually define what “completed” means for each service.

Photography: - agreed coverage completed - required event sessions covered - deliverables submitted - agreed output received

Decor: - agreed setup completed - event coverage delivered - dismantling completed if included

Catering: - agreed food/service scope delivered - agreed guest/service period fulfilled

Transport: - agreed pickup/drop services completed

Makeup: - agreed client service delivered

> Eventually maintain a: Service Completion Criteria / Verification Registry

This is a future foundation of STARVNT's Trust Layer.

## 15. SETTLEMENT

> Customer Paid ↓ Booking Executed ↓ Completion Submitted ↓ Evidence ↓ STARVNT Verification ↓ Settlement Eligible ↓ Vendor Settlement

Potentially:

> Issue Raised ↓ Settlement Hold ↓ Resolution ↓ Verification ↓ Settlement Release

This creates the foundation for the Outcome-Verified Trust Layer.

## 16. COMMERCIAL INTELLIGENCE

Once reliable financial data exists, STARVNT can calculate:

> Budget vs Committed Budget vs Projected Paid vs Due Due vs Overdue Expected Receivables Expected Settlements Event Cost Pressure

Aura+ can generate:

“Your event is currently projected to exceed the planned budget by ₹85,000.”

“Accommodation and decor are the main cost pressures.”

“You can bring the projected total within target by changing these two requirements.”

This is EVENT FINANCIAL INTELLIGENCE, not generic accounting.

## 17. VENDOR CASH-FLOW INTELLIGENCE

Future capability:

> Vendor asks: “□□□□□ 30 □□□□ □□□□ expected cash inflow □□□□?”
>
> Expected Receivables: ₹4.80L Due This Week: ₹1.20L Overdue: ₹82K Expected Settlements: ₹2.10L
>
> Aura+ can explain: “Your largest expected inflows are from three upcoming event bookings. ₹82,000 is currently overdue and should be prioritized.”

Core calculates. Aura+ explains.

## 18. NEXT BEST ACTION ENGINE

> Payment due soon → Reminder Payment overdue → Follow-up Missed commitment date → Escalation / Review Completion submitted → Verification request Completion verified → Settlement eligibility

This becomes Event Automation + Commercial Intelligence.

## 19. CUSTOMER NOTIFICATION PRINCIPLE

> Notifications must be: Useful + contextual + actionable
>
> Not: addictive + anxiety-driven + spammy
>
> Every notification should answer: What happened? Why does it matter? What can I do?
>
> Example: “Your decoration advance of ₹40,000 is due tomorrow.” CTA: View Payment

## 20. DATA OWNERSHIP PRINCIPLE

Core owns: - Event state - Booking state - Payment truth - Due state - Execution state - Completion state - Verification state - Settlement state

Aura+ owns: - Interpretation - Recommendations - Explanation - Prioritization - Next Best Action

Trust owns: - Verification - Evidence - Outcome eligibility - Settlement protection

Automation owns: - Reminders - Follow-ups - Triggered actions

No duplicate source of truth.

## 21. CUSTOMER / VENDOR / ADMIN / AURA+ VIEWS

Same Core. Different projections.

Customer sees: “What do I need to pay?”

Vendor sees: “Who owes me?”

Admin sees: “What is the financial state of this event?”

Aura+ sees: “What needs attention next?”

Same underlying truth.

## 22. DO NOT BUILD A SECOND DATABASE FOR FINANCIALS

Do not create disconnected Finance DB / Payment DB / Event DB / Booking DB without a clear relationship.

Financial objects must remain linked to Event Core.

Preserve where applicable: Event ID / Booking ID / Vendor ID / Customer ID / Quote ID / Payment ID / Execution ID / Verification ID / Settlement ID

## 23. AUDITABILITY

Every important commercial transition should be traceable:

WHO / WHAT / WHEN / WHY / SOURCE / STATUS BEFORE / STATUS AFTER

Example: Payment verified Booking: BK-1024 Amount: ₹20,000 Gateway: Razorpay Verified at: timestamp Source: webhook

## 24. MVP PRIORITY

PHASE 1 — FINANCIAL TRUTH 1. Booking amount 2. Payment amount 3. Amount paid 4. Amount due 5. Payment due date 6. Payment status 7. Server-side payment verification 8. Customer financial summary 9. Vendor receivable summary 10. Core financial state

PHASE 2 — COMMERCIAL WORKFLOW 1. Payment schedules 2. Milestone payments 3. Reminders 4. Payment links 5. Overdue states 6. Vendor receivable aging 7. Settlement eligibility 8. Settlement hold/release 9. Completion verification

PHASE 3 — INTELLIGENCE 1. Financial explanation 2. Risk detection 3. Priority detection 4. Next Best Action 5. Budget risk 6. Cost pressure 7. Receivable prioritization 8. Projected event cost 9. Vendor cash-flow insight

PHASE 4 — ADVANCED INTELLIGENCE Predictive Event Cost ↓ Budget Optimization ↓ Cash Flow Forecast ↓ Risk Prediction ↓ Automated Action Recommendation

## 25. FIRST REAL-WORLD PROOF

The first target is NOT: “Build the entire financial platform.”

The target is: Take one real event from Budget → Quote → Booking → Payment → Execution → Verification → Settlement.

> Preferably: Goa Destination Wedding — 200–500 guests
>
> Initial categories: 1. Venue 2. Accommodation 3. Decor 4. Photography/Videography 5. Makeup 6. Catering 7. Transport

## 26. SUCCESS CRITERIA

> CUSTOMER How much is my event budget? How much have I committed? How much have I paid? What is due next? What remains?
>
> VENDOR How much should I receive? Who owes me? What is overdue? Which payment needs attention? Which completed bookings are awaiting settlement?
>
> ADMIN What is the event's total commercial state? What payments are pending? Which bookings are at financial risk? Which services are completed but not verified? Which settlements are blocked?
>
> AURA+ What needs attention next? Why? What should the user do?

## 27. FINAL PRODUCT PRINCIPLE

> Customer sees simplicity. Vendor sees operations and cash flow. Aura+ sees context and recommends action. Core sees financial truth. Automation moves the workflow. Trust verifies outcomes. Memory preserves the event.

## 28. FINAL STARVNT EVENT GRAPH

> EVENT │ ├── CUSTOMER ├── REQUIREMENTS ├── BUDGET │ ↓ QUOTE ↓ BOOKING ├── PAYMENT └── EXECUTION ↓ COMPLETION ↓ VERIFICATION ↓ SETTLEMENT ↓ EXPERIENCE ↓ MEMORY ↓ INSPIRE ↺

Aura+ sits across this entire lifecycle as the intelligence layer.

## NON-NEGOTIABLE ARCHITECTURE RULES

## 1. Booking ≠ Payment 2. Payment ≠ Completion 3. Completion ≠ Verification 4. Verification ≠ Settlement 5. Aura+ never invents financial truth 6. Core remains the source of truth 7. Financial data must remain Event-contextual 8. Do not build a generic accounting/ERP product 9. Do not build aggressive debt collection 10. Do not create duplicate sources of truth 11. All payment state changes must be auditable 12. Server-side payment verification is mandatory 13. Customer UX must remain simple 14. Vendor UX can expose deeper commercial intelligence 15. Build Financial Truth first; Intelligence second; Prediction third; Automation fourth

## DEVELOPER'S ACTUAL MISSION

> The goal is not: “Build a payment dashboard.”

The goal is:

Build the commercial state of an Event into STARVNT Core so that every event can eventually be understood financially, operationally, and outcome-wise from one connected source of truth.

Eventually STARVNT should be able to answer:

> What is happening with this event? What has been paid? What is due? What has actually been delivered? What has been verified? What remains unresolved? What needs to happen next?

STARVNT does not only operate the event.

STARVNT understands the commercial state of the event.

Build the foundation first. Do not overbuild the vision in V1.